Follow the Money Without Moving the Data

Catching money laundering across banks by asking small, private, tested questions

Team Cache me if you can · LMA Group Project 2026 · Project proposal

G Sai Abhishek (2023113005) · Bhavya Ram V (2023114013) · Shreyas Deb (2023113024) · Akshat Srivastava (2023111020)

Domain	Banking: anti-money-laundering (AML) investigation.
Target user	An investigator at a bank who reviews the alerts the bank's own model isn't sure about.
Main question	A bank only sees its own customers' transactions, but laundered money hops between banks. If a bank can ask other banks a few small, private questions per alert, how much of the missing picture can it get back? And is an LLM actually good at coming up with those questions?
Capabilities	Agentic (a slow planner and a fast decision model, plus other banks as separate agents), tool calling (a small question language other banks run), reasoning (forming and testing hypotheses), RAG (the right part of the transaction graph, past questions that worked, and the relevant rules), fine-tuning (a small local planner and a small fast model).
Evaluation	IBM's AMLworld dataset, where every transaction is labelled, so every number we report is checked by code against the true answer. We compare against the dataset authors' "one bank only" model, a "see everything" upper limit, simple question pickers, and an existing LLM investigation agent.

1 Problem statement

Banks catch money laundering by watching transactions, but each bank only sees the transactions that touch its own customers. Launderers know this, so they route money through accounts at many different banks. The team behind the AMLworld dataset showed that a model looking at one bank's slice catches far less than one that sees everything [1]. Banks can't pool their data because of privacy law, but they are allowed to ask each other about specific accounts (under "314(b)" in the US and "COSMIC" in Singapore) [16–18]. Today that usually means sending over actual transaction records, which is slow and exposes a lot of customer data. A more private option is to ask a question that the other bank answers with one small, slightly noisy value, like "did this account pass on more than 80% of what it received within a day?". Nobody has measured how much such questions recover, which ones are worth asking, or whether an LLM is any good at inventing them. That last part matters: a recent LLM investigation agent made *worse* calls than the simple model it was meant to help, because it trusted its own convincing-sounding reasoning [3].

2 Why this matters, and what's missing

Reviewing alerts is the most expensive part of AML work, and the patterns that matter most (money passing straight through, going in a circle, or being split up and recombined) cross banks on purpose. If a few private questions recover most of what full data sharing would, banks get a cheaper and more private way to investigate, and the privacy cost becomes a number instead of an argument. Banks are also adopting LLM agents right now, so they need to know whether the LLM adds anything, whether its evidence can be trusted, and what each decision costs. Fraud-LLM papers rarely report this [15].

The gap, in one line: cross-bank work shares everything or nothing, LLM investigators assume they can see everything, and nobody tests an LLM's evidence before using it. Table 1 lists the closest work.

3 Research questions and what we expect

We have five questions. The first two are the heart of the project, and our main claims rest only on them.

RQ1. How much do private questions help? Of the gap between "our bank only" and "see everything", how much do a few private questions close? How does that compare with just asking for raw records?

RQ2. Is the LLM good at asking questions? Does an LLM that writes its own questions beat simpler planners? And does making every new question prove itself on past data ("back-testing") stop it from fooling itself?

RQ3. *(secondary)* **Can a small model do the routine work?** Each alert needs dozens of small decisions. Can a small, fast model make them as well as the big LLM, for much less?

RQ4. *(secondary)* **Does showing the planner the right evidence matter?** It can only read about 40 transactions. Does picking those 40 well lead to better questions?

RQ5. *(stretch)* **Is waiting worth it?** Other banks take hours or days to reply. Is it better to ask one question at a time and adapt, or several at once?

Table 2 turns these into predictions that can fail. We fix the exact thresholds on validation data and write them down before we touch the test set.

4 Our idea: following one alert

The easiest way to explain the system is to follow one alert through it (Figure 1). Say account A at Bank 7 receives 9 payments from 6 different banks within 20 hours, then sends one big payment to account X at Bank 214. Bank 7's own model gives it a score of 0.46: it genuinely can't tell.

Work	What it does	What it leaves open
AMLworld [1]	Synthetic multi-bank data; shows one-bank models do much worse	How to close that gap without sharing data
Federated AML [7–9]; private set counts [36]	Banks train jointly or compute on pooled data; two parties learn only how many items their lists share	All-or-nothing; never used for per-alert questions
Hackathon demos [22]	Agents ask other banks questions under a privacy budget	No data, baselines or evaluation
Sharma [3]; Palma [4]; FIA [5]	LLM agents investigate fraud alerts; Palma lets a calibrated formula, not the agent, make the final call	One institution with a full view; evidence isn't tested
Torres et al. [6]	LLM AML triage with subgraph evidence, citations and counterfactual checks	One bank's full view; no cross-bank questions
CAAFE [23]; POPPER [33]	Keep an LLM's ideas only if they pass a statistical test	Offline features or science hypotheses
LivingRAG [24]; ReasoningBank [34]; FinFRE-RAG [28]	Reuse past reasoning and strategies; retrieve similar fraud cases	Text or web tasks; reusing by account would leak here
SubgraphRAG [25]; G-Retriever [26]	Pick the useful, connected part of a graph for an LLM	Text knowledge graphs, not transaction graphs
System One models [29–31]; Conformal Cascade [35]	Small models that give a fixed-format answer with a confidence, and pass hard cases up	Never tested inside a multi-agent system
LLM vs classical planners [10–13]	Classical planners often match LLMs at choosing what to ask	Not tested on money trails across banks

Table 1: Closest prior work and what it leaves open.

	We expect	We're wrong if
Primary: tested in this order, with Holm correction		
H1	With a privacy budget of ε=2 per alert, questions close at least 30% of the gap (Brier score, Section 9). Step 3 explains why ε=1 is too tight	The gain could be zero (95% interval), or it's under 30%
H2	Questions get at least 70% of the benefit of sharing raw records, while sharing none	The ratio is under 70%
H3	The LLM writing its own questions is at least as good as the best simple planner *and* the no-LLM question enumerator, and better than itself limited to a fixed list	Clearly worse than either, or no better than the fixed list
H4	Back-testing matters: without it results get worse; with it, under 10% of accepted questions turn out useless on later data	No difference, or over 10% useless
Secondary		
H6	Smart retrieval shows the planner at least 15 points more of the real laundering pattern, and improves results	Under 15 points, or no effect
H7	A calibrated formula making the final call beats the LLM deciding (repeating Palma [4] in our setting)	No measurable difference
Exploratory: reported without significance claims		
H5	The fast model handles most routine decisions within its stated accuracy bound, at least 10× faster, with no loss in detection	Below the bound, under 10×, or worse detection
H8	Asking two questions at once, then one more, keeps 80% of the benefit in 60% of the time	Either number is missed

Table 2: Our predictions, in three tiers. Only the primary tier supports the main claims.

Step 1: deciding which alerts we look at. For each of the 30 biggest banks, we keep only the transactions that bank would really see and train the dataset authors' one-bank model on it (graph features + gradient-boosted trees [1,2]). An alert is one transaction this model scores, and its label is that transaction's laundering flag. Instead of hand-picking an "unsure" score range, we use conformal prediction: an alert reaches our system only when the model can't confidently rule out either answer. We use the version that bounds misses among actual launderers (laundering is rare, so the plain version could hit its target by calling almost everything clean), plus selective risk control [38] to bound misses among the alerts the model clears on its own. The data drifts from day to day, so we weight recent days more [39]. Under drift these bounds only hold approximately, so we always report the miss rate we actually get.

Step 2: showing the planner the right evidence. A's neighbourhood can contain thousands of transactions,

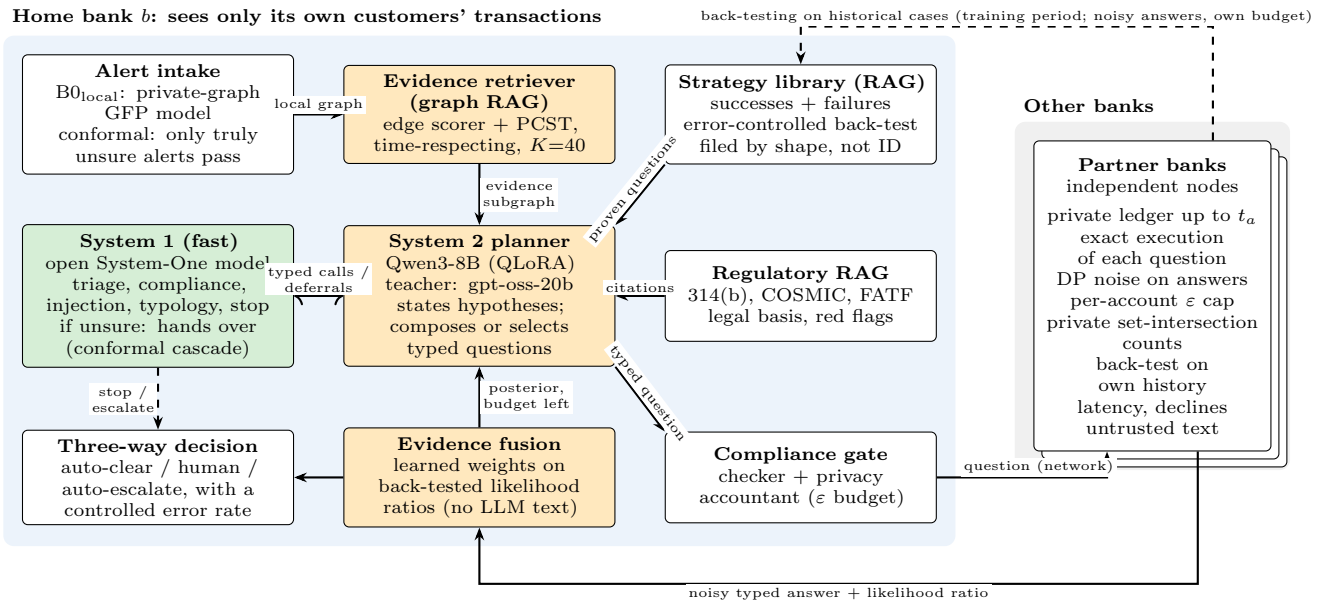


Figure 1: System overview. Orange boxes are our main contributions; the green box is the fast decision model. The LLM never makes the final call: only the other banks' answers, weighted by how reliable they proved on past data, reach the decision. No transaction record leaves another bank.

but an 8B model can only really reason over about 40. A small scorer in the style of SubgraphRAG [25], trained on the training period, rates each transaction by how likely it is to be part of a laundering pattern. A graph step from G-Retriever [26] then joins the top ones into one connected, time-ordered money trail, instead of 40 scattered rows.

Step 3: asking a question, not for data. The planner says what it suspects ("money is passing straight through A") and picks or writes a question for one other bank, in a small question language that bank can run exactly. Questions are templates whose slots are relative to the alert ("the alert account's payers", "the day the transfer arrived"), never fixed accounts or dates; that is what lets us back-test them later. A question can ask about an account's behaviour, ask which of a short list of banks the money went to next, or count how many accounts two banks' lists share using a private set-intersection count [36]. (Plain hashing isn't private, since a bank could hash every possible account ID and match them back. We simulate the protocol's noisy output rather than build the cryptography.)

The other bank answers with one value and adds a little random noise first. This is differential privacy [32], and it's worth being clear about what it protects and what it costs:

- **What's protected is one transaction.** Adding or removing any single transaction changes the chance of any answer by at most a factor of e^{ε}. That's weaker than protecting a whole customer, so we also cap the total ε spent on each partner account across all alerts and all asking banks, for the whole test period.
- **The noise caps how much one answer can tell us.** An answer bought with budget ε_q can shift the odds of laundering by at most e^{ε_q} for a yes/no answer, or $e^{\varepsilon_q \Delta}$ for a count whose typical values for launderers and clean accounts differ by Δ. So at ε=1 per alert, yes/no answers alone could never move our 0.46 alert above about 0.70. That's why H1 uses ε=2, and why we draw this ceiling on every privacy plot. We test ε = 0.5, 1, 2, 4 and no noise.
- **Replies can carry one short text note**, as real 314(b) replies do. It never enters the decision, but the planner reads it, so it's where we test hidden (injected) instructions.

In our example, the planner takes a proven question from the library and asks Bank 214, spending ε=1.0: *"Did X pass on at least 80% within 24 hours, and if so, which of these 4 banks got most of it?"* Back-testing gave this question a likelihood ratio of 2.6, just under the ceiling of $e^1 \approx 2.7$. The answer is yes, mostly to Bank 88. A's probability goes from 0.46 to **0.69**.

Step 4: making every question earn its weight. Before a question can affect any decision, we run it on past alerts where we know who was laundering and measure its *likelihood ratio*: how much more often "yes" shows up for launderers than for clean accounts. Questions that tell us nothing are thrown out, so the LLM can't lean on clever-sounding but useless ones. Two things can go wrong here, and we handle both:

- **Luck.** If the LLM invents 2,000 questions and we accept anything that passes a 5% test, about 100 useless ones get in by chance. So, following POPPER [33], each question is tested with *e-values*, an evidence score that stays valid even if we keep adding data and stop whenever we like. An e-value version of false-discovery-rate control [37] then keeps useless questions under 10% of the library.
- **Cheating.** A real bank can't back-test on every bank's history with perfect labels; that would be the "see

everything" world sneaking back in. So all main results use *realistic* back-testing: labels come only from the home bank's own past alerts, and other banks answer historical questions with the same noise as live ones, charged to a separate calibration budget ε_{cal} that we report. Exact back-testing with every bank's labels is only an ablation, to show what realism costs. Past alerts also go through the same Step 1 filter, so ratios are measured on the same kind of hard case.

Back to A. Now the LLM writes a brand-new question for Bank 88, spending ε=0.8: *"Of the accounts that got money from the bank that received A's transfer, on the day it arrived, how many then paid any of A's payers within 72 hours?"* This is a private set count. Launderers typically give 2 and clean accounts 0, so the ceiling is $e^{0.8\times 2} \approx 5.0$; the question passes back-testing with a ratio of 4.4. The answer is 2, and A's probability rises to **0.91**. The money went in a circle across three banks, invisible to Bank 7 alone, and we found it with two questions, 90% of a budget of ε=2, and zero records shared. A planner stuck with a fixed list has no question that checks whether money came back to A's own payers.

Step 5: remembering what worked. Following ReasoningBank [34], the library keeps short strategies rather than transcripts, learned from both successes and failures: "money in from many banks, then out in one big transfer: first ask whether the receiver passed it on within a day". Failure notes ("don't bother asking this here") come only from back-test statistics over many alerts, never from one noisy answer. Everything is filed by the *shape* of the situation, never by account ID, because the same laundering rings reappear later in AMLworld and filing by account would just memorise known criminals. We run both versions to show this. Since we only know whether a case succeeded from its label, the library is built on training and validation data and frozen before testing. Letting it keep learning from delayed test labels is a separate, exploratory run.

Step 6: letting a fast model do the small stuff. Each alert needs 20–50 small decisions: is this account worth asking about, which pattern is this, does this rule apply, is there a hidden instruction in this reply, should we stop? These go to a "System One" style model that answers a fixed-format question in one pass, with a confidence score, and never writes free text. We test a training-free one (SemIf [30]), a small trained one (Laya [31]) and our own fine-tuned 4B model. Their training labels come from the simulator: "worth asking" is true if the account is in a real laundering pattern, "which pattern" is the pattern type, and "should we stop" is true if no library question would change the final call. To decide when to trust the fast model, we use Conformal Cascade [35]: the fast model gives a set of plausible answers, we accept it if the set has exactly one answer, and otherwise the 8B model decides. With two tiers this bounds the error at 2α for a chosen α (the tighter bound of α needs an extra condition, which we check on validation). If the 8B model is unsure too, we take the safe default: keep investigating, or hand the case to a human.

Step 7: making the final call with maths, not words. The final probability starts from the bank's own score and is updated by each answer according to its back-tested likelihood ratio. Two questions can overlap (both really asking about the same transfer), so simply adding up their evidence would make us overconfident. Instead, we learn weights on validation data, fitted together with the bank's own score so nothing gets counted twice. Weights are learned per question family (questions from the same template); a brand-new LLM question takes its family's weight, or a shrunken one if it has no family. The LLM's text never enters this calculation, so a convincing but wrong argument can't change the outcome.

The agent stops when the fast model says the case is clear or the budget runs out. Following FCAC [40], each case then ends as auto-clear, send to a human, or auto-escalate. FCAC's error control relies on randomly audited past cases and an assumption about how fast risk changes, so we simulate audits (a random 5% of past cases get their label after a delay) and call the result "controlled under FCAC's assumption" rather than guaranteed. Each calibrated step is calibrated only on past cases that went through the same earlier steps.

Finally, each other bank is simulated as its own node: it usually replies within a day (sometimes up to 4), may decline, and times out after 5 days. That is what lets us study RQ5.

5 Why we need each capability

Table 3 shows why our problem needs each capability and how we'll know it works. Each one is also switched off in an ablation (Section 8).

6 What's new

Each building block exists somewhere already; what's new is how we put them together and what we measure. We give (1) the first privacy-vs-detection curve for cross-bank questions, including set counts for following money, compared against sharing raw records; (2) LLM-invented questions that must pass error-controlled back-testing before they count, which directly targets the failure in [3]; (3) a strategy library filed by situation rather than account, with a test showing why filing by account cheats; (4) the first test of open System One models inside a multi-agent system, with a hand-off that has a stated accuracy bound; and (5) evidence retrieval for one bank's transaction graph that keeps the trail connected and in time order. If the LLM turns out no better than simple planners, or questions recover little, that's still a useful, measured result.

Capability	Why we need it, and what it does here	How we measure it
Agentic	Investigating means asking, reading the answer and deciding what to ask next, under a budget, while other banks act on their own. A slow planner, a fast decision model and bank agents work together and can hand over to a human	Gap closed per unit of privacy; time to decision; cost per alert
Tool calling	The only way to get evidence is a question another bank runs, and every question spends real budget. Questions go through a rules and privacy checker first	Questions that parse and run; recovery from timeouts and refusals; unsafe actions proposed
Reasoning	Picking a question means guessing what's going on and testing it. The planner states hypotheses; back-testing checks its new questions	New questions that pass; ones that later fail; stated vs real confidence; gain over the enumerator
RAG	The planner sees only a slice of the graph, should reuse what worked, and the compliance gate must find which legal basis applies (314(b) and COSMIC differ)	Pattern edges shown; library reuse; correct citations vs the whole rules corpus in the prompt
Fine-tuning	Banks need this on their own hardware, cheaply, with honest confidence. The 8B planner learns from gpt-oss-20b, using only teacher questions that passed back-testing and moved towards the right answer; the 4B fast model learns calibrated confidence	Planner vs teacher; fast model's accuracy, calibration, speed and share of decisions kept

Table 3: The five capabilities.

7 Data

We use **AMLworld** [1], synthetic but realistic banking data from IBM (Kaggle, CDLA-Sharing licence). Because it comes from a simulator, every transaction has a correct label and we know which laundering pattern it belongs to. Our main set is **HI-Medium**: about 32 million transactions over 16 days, with about 35,000 laundering transactions. It's exactly the setting where the authors compared one-bank and shared models, so we can check our baseline against theirs. We develop on HI-Small (5 million transactions).

We split by time: days 1–9 for training, 10–12 for validation and 13–16 for testing. An alert is dropped if its laundering pattern spans a split, or if its 72-hour evidence window starts before the split. Tools only see data up to the moment of the alert, and account IDs are replaced by random aliases in anything the LLM reads. AMLworld builds laundering from a fixed set of pattern types (fan-in, fan-out, cycles, scatter-gather and so on), and our questions target similar shapes. To check we aren't just reverse-engineering the simulator, we also rerun the pipeline with cycles removed from all training and back-testing, and test on cycle alerts.

Resource	Size	Notes
Test alerts the bank is unsure about	2,500–4,000	At least 375 real laundering cases; Step 1 decides the count, so we confirm it at gate G1
Training / validation alerts	$\approx$20,000 / $\geq$1,000	Back-testing, training, fixing thresholds
Expert question list	$\approx$60	Written by 2 teammates from FATF red-flag reports; used by the simple planners
Small-decision dataset	$\approx$50,000	Labels from the dataset's labels and pattern files
Rules corpus	$\approx$40 red flags	314(b) rules, FinCEN fact sheet, MAS COSMIC guidelines, FATF red flags [16–19]
Injection test set	4 types $\times$ 100	Fake instructions hidden in other banks' reply notes

Table 4: Datasets and test resources.

8 Baselines and ablations

Every system runs on the same alerts, the same simulated banks and the same privacy rules. Each row changes one thing.

System	What it is	What it tells us
One bank only	The authors' one-bank model, no questions [1]	Where we start
See everything	The same model with every bank's data (published: 63.23 F1 on HI-Small [1,2])	The best possible case
Share raw records	Ask three banks for the actual transactions, no noise	What full sharing buys (H2)
Simple planners	Pick from the expert list at random, in a fixed order, by expected information gain, or with a learned policy	Is an LLM needed? (H3)
Enumerator (no LLM)	Every question the language can express up to a small size, back-tested the same way, picked by information gain	Does the LLM add more than a bigger question set? (H3)
LLM, fixed list	Our planner, but it can't write new questions	Is writing questions worth it? (H3)
No back-testing	The planner judges its own evidence	Does back-testing matter? (H4)
LLM decides	Same tools, but the LLM gives the final verdict, as in [3]	Formula vs LLM (H7)
Fast-model options	8B LLM writing JSON vs SemIf vs Laya vs our 4B model	RQ3 (H5)
Evidence views	Biggest 40 transactions vs learned scorer vs scorer + connecting step vs everything in a long prompt	RQ4 (H6)
Other switches	Realistic vs exact back-testing; no hypotheses ("act-only"); rules retrieved vs whole corpus in the prompt; library by account vs by shape; ε sweep; per-account cap on vs off; held-out cycles; one-at-a-time vs all-at-once; refusals and timeouts; shuffled answers (the gain should vanish); fixed 0.15–0.85 range vs conformal; adding evidence vs learned weights; optional RL planner [27,41]	Privacy, robustness, leakage, generalisation, calibration

Table 5: Systems we compare.

9 Metrics

Every main number is computed by code against the true labels; no LLM grades anything. Our headline metric is the **Brier score**, the squared gap between our predicted probability and what actually happened. Lower is better: it rewards being confidently right and punishes being confidently wrong. **Gap closed** turns it into a percentage, where 0% is no better than one bank alone and 100% is as good as seeing everything:

$$\text{Gap closed} = \frac{\text{Brier(one bank)} - \text{Brier(our system)}}{\text{Brier(one bank)} - \text{Brier(see everything)}}$$

Area	What we report
Detection	Brier, gap closed, balanced accuracy, precision–recall; launderers caught when investigators can review a fixed number of cases a day; correct calls flipped to wrong (and back); auto-decision error rates vs their targets
Privacy, cost	Gap closed per unit of ε, with the ceiling on the same plot; per-alert and per-account ε; calibration budget ε_{cal}; records shared; questions per alert; LLM calls replaced; GPU-seconds per alert
Speed, tools	Simulated time to a decision; questions that parse and run; recovery after timeouts, refusals and broken replies
Reasoning	New questions that pass; passed questions that turn out useless on test data (ratio ≤ 1, among questions asked $\geq$30 times); stated vs real confidence; whether questions match the stated hypothesis (2 teammates check 100 steps)
RAG	Real pattern edges among the 40 shown; whether they form one trail; library reuse; whether cited rules exist and apply
Fine-tuning	Planner vs teacher; fast model's accuracy, calibration and speed per decision type; share kept; accuracy vs its bound
Safety	Rule-breaking actions proposed; actions triggered by injected text; laundering cases wrongly cleared

Table 6: Metrics. The "see everything" row appears in every results table, so all numbers can be read as a share of what's possible.

Keeping ourselves honest. Confidence intervals come from a cluster bootstrap: related accounts and patterns are grouped and resampled together 10,000 times. Before touching the test set we write down every threshold and the order we test H1–H4 in, with Holm correction; secondary hypotheses are corrected separately, and exploratory ones get no significance claims. We only call two planners "as good as" each other if their difference is under half the gap between the best simple planner and random guessing. Gap closed is a ratio, so if its denominator turns out tiny we report raw Brier differences instead. With 4,000 test alerts we can detect Brier improvements of about 0.011. Every error is tagged with the step where it went wrong, and we write up ten example cases, good and bad.

10 Safety plan

The data is synthetic and the rules corpus is public, so no real people are involved. No records leave a bank: replies are single noisy values, the budget is tracked per alert and capped per account, back-testing is charged to its own budget, and set counts never reveal either bank's list. Every question and action (like drafting a suspicious-activity report) goes through a rules checker that blocks anything not explicitly allowed. Replies are treated as data, not instructions [14]: the fast model screens their notes for hidden instructions, and we report how often injected text gets the agent to propose a bad action, with and without the checker. Finally, a system that learns which questions clear an account could help criminals, so we release the code and the simulator but not the trained weights.

11 Compute plan

Everything runs locally on one NVIDIA RTX 5070 Ti (16 GB) with 32 GB of RAM, for $0. The 8B model runs on vLLM in 4-bit, fine-tuning uses QLoRA [20,21] through Unsloth, and the teacher (gpt-oss-20b) also fits on the GPU. The one-bank baseline uses IBM's free snapml library; if its licence blocks publishing, we rebuild its graph features ourselves (30–50 hours).

Item	Estimate	Why
Data prep, one-bank models, planners	40–150 CPU-hours	31 graph-feature runs, in daily chunks
Back-testing	90–320 CPU-hours	≈2,000 LLM questions plus enumerated ones, on ≈20,000 alerts; realistic and exact versions; held-out rerun
Fine-tuning	25–45 GPU-hours	QLoRA on 8B and 4B; small scorer
Running the agents	75–170 GPU-hours	≈28,000 investigations plus ≈4,000 teacher runs, 8–20 seconds each
Total GPU	**100–215 h (expect ≈115; 250 budgeted)**	Even the worst case fits; the optional RL planner [27,41] runs only if we're near the expected figure

Table 7: Compute budget. One final run on the test set; all tuning happens on validation data.

12 Plan, scope and risks

We build what every experiment needs first: data, bank views, baselines, the simulated banks, the question language and back-testing. None of this needs a GPU, so we find out early whether questions help at all. The team splits into four streams: data and baselines; simulated banks and simple planners; LLM, fine-tuning and retrieval; rules, tests, statistics and write-up.

We plan to build the whole system, but the project doesn't stand or fall on all of it:

Tier	What it contains
Core	Baselines; simulated banks with noisy answers, set counts and the privacy accountant; question language; realistic back-testing; LLM, simple and enumerator planners; evidence fusion; conformal intake. Answers RQ1–2 (H1–H4). *Must be done by G5; the main claims rest only on this.*
Secondary	Graph-RAG retriever (H6); LLM-decides baseline (H7); strategy library; fine-tuned planner and 4B model; held-out cycles; regulatory RAG; injection tests. *Planned in full; if late, we report what's finished.*
Stretch	Conformal Cascade with SemIf and Laya (H5); FCAC three-way decision; one-at-a-time vs all-at-once (H8); learning from delayed test labels; RL planner. *Only if the core is on time at G4.*

Table 8: Scope tiers. A dropped stretch item falls back to a simpler version of the same step (the 8B model makes all small decisions; one calibrated threshold replaces the three-way decision), so the pipeline keeps its shape.

Gate	When	What must be true to continue
G0	Before approval	Only this proposal, data download and install checks. Nothing is built before the TA signs off
G1	Weeks 1–2	Our one-bank model matches the published per-bank results; enough test alerts expected (judged from validation)
G2	Weeks 2–3	With realistic back-testing at ε=2, expert questions help by at least 0.01 Brier on validation and stay under the ceiling (a check on our privacy code). If not, we report the curve, loosen the budget and tell the TA
G3	Weeks 3–4	At least 20 expert questions pass back-testing; the enumerator and fast models run on our GPU; library frozen; plan locked
G4	Week 9	Core done on validation; fine-tuned models and retrieval demo; pick which stretch items to attempt
G5–G6	Weeks 14, 16	Final test run and report; presentation

Table 9: Milestones. Any change to the primary hypotheses after G3 goes back to the TA.

Risk	What we do about it
Noisy answers carry too little information	We know the ceiling in advance; counts carry more than yes/no, so the planner can favour them; G2 checks early; the curve is a result either way
Realistic back-testing is too noisy	Compare with exact back-testing at G2; raise ε_{cal} and report it
Weights and conformal bounds drift	Recalibrate on the latest validation days; retire questions that stop working; report achieved vs target rates
The LLM's questions add nothing	We still get a clear, measured "no", with costs attached
The system memorises known criminals	Library filed by shape; random account aliases; shuffled-answer test
Tools don't work out (new fast models, snapml licence, weak teacher)	Checked at G0 and G3; our own 4B model is the main fast model; rebuild snapml's features if needed; try Qwen3-14B as teacher
Running out of time	Drop stretch items in this order: RL planner, learning from test labels, the RQ5 study, SemIf and Laya, Conformal Cascade, FCAC. The core is never cut

Table 10: Main risks.

References

[1] E. Altman et al. Realistic Synthetic Financial Transactions for Anti-Money Laundering Models. NeurIPS 2023 Datasets and Benchmarks; arXiv:2306.16424.

[2] J. Blanuša et al. Graph Feature Preprocessor: Real-time Subgraph-based Feature Extraction for Financial Crime Detection. arXiv:2402.08593, 2024.

[3] R. Sharma. Toward Auditable Fraud Detection: Combining Graph Features, Model Explanations, and Agentic Case Investigation. arXiv:2607.19266, 2026.

[4] D. Palma, K. B. Kim, Z. Han, A. Dsouza, Z. Liu. Business Compromise Detection with Agentic AI and LLM-driven Knowledge Discovery. EMNLP 2026; arXiv:2609.32643.

[5] FIA Framework: A Large Language Model-Based Approach for Credit Card Fraud Investigations. arXiv:2506.11635, 2025.

[6] D. Torres, W. Cheng, K. Hu. Explainable AML Triage with LLMs: Evidence Retrieval and Counterfactual Checks. arXiv:2604.19755, 2026.

[7] Bank for International Settlements. Project Aurora: the power of data, technology and collaboration to combat money laundering across institutions and borders. 2023.

[8] Z. Tian, Y. Ding, W. Qu et al. Towards Collaborative Anti-Money Laundering Among Financial Institutions. WWW 2025; arXiv:2502.19952.

[9] Ceydeli, Wang, Atasu. Federated Cross-Client Subgraph Pattern Detection. arXiv:2605.06433, 2026.

[10] D. Choudhury et al. BED-LLM: Intelligent Information Gathering with LLMs and Bayesian Experimental Design. arXiv:2508.21184, 2025.

[11] GraphDx: A Cost-Aware Knowledge-Enhanced Multi-Agent Framework for Sequential Diagnosis. ACL Findings 2026; arXiv:2607.15280.

[12] Perets et al. Zero-Shot Active Feature Acquisition via LLM-Elicitation. arXiv:2606.18933, 2026.

[13] T. LaRock et al. NOL*: reinforcement learning for exploring partially observed networks. Applied Network Science, 2020; arXiv:2001.07607.

[14] E. Debenedetti et al. Defeating Prompt Injections by Design (CaMeL). arXiv:2503.18813, 2025.

[15] Operational Evidence Gaps for LLMs in Fraud Detection and Trust-and-Safety Workflows (FORTE). arXiv:2607.13078, 2026.

[16] 31 CFR §1010.540: Voluntary information sharing among financial institutions. U.S. Code of Federal Regulations.

[17] FinCEN. Section 314(b) Fact Sheet. 12 June 2026.

[18] Monetary Authority of Singapore. Notice FSM-N02 and Guidelines on COSMIC. 2024.

[19] FATF. Money laundering red-flag indicator reports.

[20] E. J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022.

[21] T. Dettmers et al. QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS 2023.

[22] lablab.ai hackathon submissions: Federated analytics for cross-bank AML; AEGIS autonomous financial-crime investigation. 2026.

[23] N. Hollmann, S. Müller, F. Hutter. Large Language Models for Automated Data Science: Introducing CAAFE for Context-Aware Automated Feature Engineering. NeurIPS 2023.

[24] Y. Cui, Z. Zhang, Q. Liu. LivingRAG: Augmenting Graph RAG with Experience. arXiv:2608.25960, 2026.

[25] M. Li, S. Miao, P. Li. Simple Is Effective: The Roles of Graphs and Large Language Models in Knowledge-Graph-Based Retrieval-Augmented Generation (SubgraphRAG). ICLR 2025; arXiv:2410.20724.

[26] X. He et al. G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering. NeurIPS 2024; arXiv:2402.07630.

[27] V. Gandhi et al. GRASP: GRanularity-Aware Search Policy for Agentic RAG. EMNLP 2026 Findings; arXiv:2607.10463.

[28] X. Tan, Y. Ma, X. Zhang. Understanding Structured Financial Data with LLMs: A Case Study on Fraud Detection (FinFRE-RAG). ACL 2026; arXiv:2512.13040.

[29] TypeSafe AI. Jev, a System One decision model (commercial; mentioned for context, not used). September 2026.

[30] SemIf: typed decisions from a frozen language model's answer probabilities (MIT). Open-source software, 2026.

[31] Laya: small non-autoregressive typed decision model with calibrated outputs (Apache-2.0). Open-source software, 2026.

[32] C. Dwork, A. Roth. The Algorithmic Foundations of Differential Privacy. Foundations and Trends in Theoretical Computer Science, 2014.

[33] K. Huang et al. Automated Hypothesis Validation with Agentic Sequential Falsifications (POPPER). ICML 2025; arXiv:2502.09858.

[34] S. Ouyang et al. ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory. ICLR 2026; arXiv:2509.25140.

[35] Y. Dou, S. Lian, S. Li. Conformal Cascade: Distribution-Free Accuracy Guarantees for Multi-Tier LLM Inference. arXiv:2607.25018, 2026.

[36] C. Purcell et al. Split, Count, and Share: A Differentially Private Set Intersection Cardinality Estimation Protocol. PMLR vol. 216 (UAI 2023).

[37] R. Wang, A. Ramdas. False Discovery Rate Control with E-values. Journal of the Royal Statistical Society Series B, 2022.

[38] Y. Xu, W. Guo, Z. Wei. Selective Conformal Risk Control. arXiv:2512.12844, 2025.

[39] Non-exchangeable Conformal Prediction for Temporal Graph Neural Networks. arXiv:2507.02151, 2025.

[40] J. Deng. When Can Fraud Operations Authorize Automation? A Decision-Support Framework for Fresh Audit Evidence and Review Workload (FCAC). arXiv:2608.08577, 2026.

[41] S. Hu et al. Optimizing Agentic Reasoning with Retrieval via Synthetic Semantic Information Gain Reward (InfoReasoner). ICML 2026; arXiv:2602.00845.